

Lab 1: OLS by Hand, in Scalar Form

SOCIOL 723, Social Statistics II, Thursday, August 27

Wenhao Jiang

Table of contents

1	Goals	2
2	Getting set up	2
2.1	Packages	2
2.2	Working reproducibly	2
3	The data	3
4	OLS by hand, in scalar form	5
4.1	The slope and intercept from sums	5
4.2	The slope is a weighted average of unit slopes	6
4.3	Residual mechanics	6
4.4	s^2 and the classical standard error	6
5	Simulation: what the error assumption does	7
5.1	When the average of Y really is a line	7
5.2	When the CEF is not a line: uncorrelated is not mean-independent	7
5.3	Heteroskedasticity does not bias the slope	8
6	The multiple regression, and fit	8
6.1	The sum-of-squares decomposition	9
7	Frisch–Waugh–Lovell	9
7.1	The added-variable plot	10
8	Omitted variable bias, numerically	11
9	Interpretation: functional form	12
9.1	Logs	12
9.2	Quadratics	13
9.3	Interactions	13
9.4	Saturating the key covariate	13

10 Exercises	14
11 Session information	15

1 Goals

By the end of this lab you should be able to:

1. Organize an analysis in RStudio and Quarto so that it is reproducible.
2. Compute $\hat{\beta}_0$ and $\hat{\beta}_1$ from the scalar formulas with nothing but `sum()`, and verify that they match `lm()`.
3. Verify the mechanical residual properties, and compute s^2 and the classical standard error by hand.
4. Use **simulation** to see what the assumption $E[\epsilon_i | X_i] = 0$ does and does not deliver.
5. Verify the Frisch–Waugh–Lovell theorem and the omitted-variable-bias formula in real GSS data.

Everything below is scalar arithmetic: sums, means, and `lm()`. No matrix computations appear anywhere in this lab.

2 Getting set up

2.1 Packages

```
library(dplyr)      # data manipulation
library(ggplot2)    # graphics
library(broom)      # tidy() / augment() for model objects
library(modelsummary) # regression tables
```

If any of these are missing, install them once with `install.packages(c("dplyr", "ggplot2", "broom", "modelsummary"))`.

2.2 Working reproducibly

Three habits, which I will assume for every problem set:

- **Use an RStudio Project.** File > New Project. Never use `setwd()`; paths should be relative to the project root.
- **Write in Quarto (.qmd) or RMarkdown (.Rmd).** Your submission should be a rendered PDF produced from a source file that I could run myself.
- **Set a seed** before anything random: `set.seed(1)`.

A useful discipline: restart R (Session > Restart R, or Cmd/Ctrl+Shift+F10) and re-render from a clean session before you submit. If it does not run from scratch, it is not reproducible.

3 The data

We use an extract from the **General Social Survey** (NORC) cumulative file, pooling the 2010, 2012, 2014, 2016, 2018 and 2022 waves. The sample is restricted to respondents working full time, aged 25–64, with positive personal income and non-missing values on the analysis variables.

```
gss <- readRDS("../Data/gss_earnings.rds")
dim(gss)
#> [1] 3509 23
str(gss, give.attr = FALSE)
#> 'data.frame': 3509 obs. of 23 variables:
#> $ year : int 2010 2010 2010 2010 2010 2010 2010 2010 2010 2010 2010 ...
#> $ id : int 1 13 20 21 29 35 38 42 44 50 ...
#> $ llearn : num 10.66 9.87 10.66 10.26 10.46 ...
#> $ realrinc: num 42735 19425 42735 28490 34965 ...
#> $ educ : num 16 12 16 12 16 15 16 18 17 12 ...
#> $ degree : Ord.factor w/ 5 levels "< HS"<"HS"<"Junior college"<...: 4 2 4 2 4 2 5 5 5 2 .
#> $ ba : int 1 0 1 0 1 0 1 1 1 0 ...
#> $ exper : num 9 40 13 31 11 36 31 9 2 22 ...
#> $ age : num 31 58 35 49 33 57 53 33 25 40 ...
#> $ female : int 0 0 0 0 1 0 0 0 0 1 ...
#> $ black : int 0 1 0 0 0 0 0 0 0 0 ...
#> $ otherace: int 1 0 0 0 0 0 0 1 1 1 ...
#> $ wordsum : num 6 5 6 6 8 7 3 8 3 2 ...
#> $ pareduc : num 8 16 12 12 12 12 18 12 18 6 ...
#> $ prestige: num 60 24 33 49 47 41 41 61 46 30 ...
#> $ sei : num 76.3 20.7 46.2 45.4 38.8 54.6 54.6 80.9 61.4 20.1 ...
#> $ hours : num 55 40 50 40 40 60 40 35 60 50 ...
#> $ childs : num 0 2 0 0 0 2 2 0 0 2 ...
#> $ evermar : int 0 1 0 1 0 1 1 0 0 1 ...
#> $ region : Factor w/ 9 levels "New England",...: 2 2 2 2 2 2 2 2 2 1 ...
#> $ stratum : int 2240 2242 2243 2243 2244 2245 2245 2246 2246 2247 ...
#> $ psu : int 22401 22421 22431 22431 22442 22451 22452 22461 22461 22471 ...
#> $ wt : num 1.127 0.64 0.447 0.554 0.39 ...
```

Variable	Meaning
<code>llearn</code>	log of personal income, constant dollars (<code>realrinc</code>)
<code>educ</code>	years of schooling completed
<code>exper</code>	potential experience, $\max(\text{age} - \text{educ} - 6, 0)$
<code>female</code> , <code>black</code> ,	indicator variables
<code>otherace</code>	
<code>wordsum</code>	score on a 10-word vocabulary test (proxy for verbal ability)
<code>pareduc</code>	highest education of either parent, in years

Variable	Meaning
prestige, sei	occupational prestige and socioeconomic index
hours	usual hours worked last week
region	Census division (9 categories)
stratum, psu	GSS sampling stratum and primary sampling unit (used in Lab 2)
wt	post-stratification sampling weight

i Note

The script that builds this file from the raw cumulative file is in `Data/build_gss_extract.R`. You do not need to run it.

A first look:

```
gss |>
  summarise(across(c(llearn, educ, exper, wordsum, pareduc, hours),
                    list(mean = mean, sd = sd), .names = "{.col}_{.fn}")) |>
  t() |> round(2)
#>           [,1]
#> llearn_mean  9.95
#> llearn_sd    0.93
#> educ_mean   14.48
#> educ_sd      2.82
#> exper_mean  23.18
#> exper_sd    11.22
#> wordsum_mean 6.33
#> wordsum_sd   1.91
#> pareduc_mean 13.20
#> pareduc_sd   3.61
#> hours_mean  45.49
#> hours_sd    11.33
```

```
gss |>
  group_by(educ) |>
  summarise(m = mean(llearn), n = n()) |>
  filter(n >= 20) |>
  ggplot(aes(educ, m)) +
  geom_point(aes(size = n), colour = "navy", alpha = .7) +
  geom_smooth(method = "lm", se = FALSE, colour = "firebrick", linewidth = .6) +
  labs(x = "Years of schooling", y = "Mean log earnings", size = "n") +
  theme_minimal(base_size = 10)
```

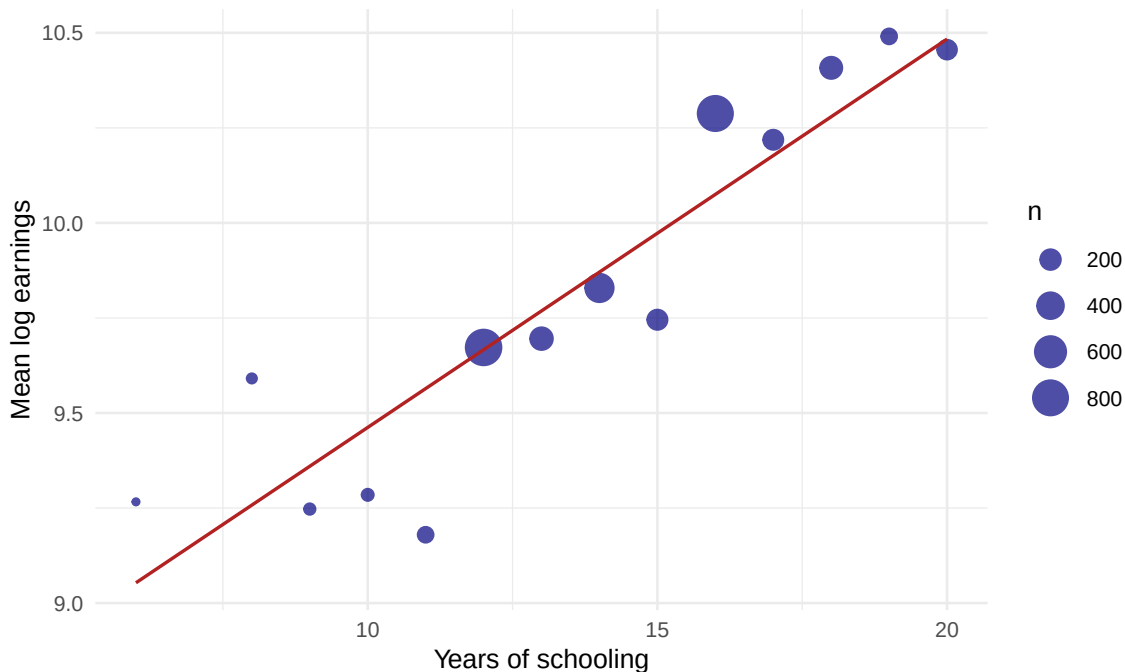


Figure 1: The CEF of log earnings given years of schooling, GSS 2010–2022.

The points are the **conditional expectation function** estimated cell by cell (this is what a *saturated* regression would fit). The line is the **best linear approximation** to it. Notice that the CEF is not exactly linear; it is convex above roughly 12 years, but the line summarizes it reasonably well.

4 OLS by hand, in scalar form

4.1 The slope and intercept from sums

The whole estimator is two lines of arithmetic: $\hat{\beta}_1 = \text{Cov}_n(X, Y) / \text{Var}_n(X)$ and $\hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \bar{X}$.

```
x <- gss$educ
y <- gss$lnearn
n <- length(y)

b1 <- sum((x - mean(x)) * (y - mean(y))) / sum((x - mean(x))^2)
b0 <- mean(y) - b1 * mean(x)
round(c(b0 = b0, b1 = b1), 4)
#>      b0      b1
#> 8.2307 0.1187
```

Compare with `lm()`:

```
fit_biv <- lm(lnearn ~ educ, data = gss)
round(coef(fit_biv), 4)
```

```
#> (Intercept)      educ
#>      8.2307      0.1187
```

Identical, and identical to the numbers on the lecture slides. `lm()` is doing exactly this arithmetic (in a numerically fancier way), nothing more.

4.2 The slope is a weighted average of unit slopes

Lecture claimed $\hat{\beta}_1 = \sum_i w_i (Y_i - \bar{Y}) / (X_i - \bar{X})$ with $w_i = (X_i - \bar{X})^2 / \sum_j (X_j - \bar{X})^2$. Verify it, and look at who carries the weight:

```
w <- (x - mean(x))^2 / sum((x - mean(x))^2)
bi <- ifelse(x == mean(x), 0, (y - mean(y)) / (x - mean(x)))
c(weighted_average = sum(w * bi), b1 = b1)
#> weighted_average      b1
#>      0.1187      0.1187

## total weight carried by each value of educ
tapply(w, x, sum) |> round(3)
#>      0      2      3      4      5      6      7      8      9     10     11     12     13
#> 0.022 0.017 0.019 0.012 0.010 0.054 0.020 0.045 0.042 0.032 0.041 0.183 0.020
#>     14     15     16     17     18     19     20
#> 0.004 0.002 0.066 0.042 0.108 0.069 0.193
```

Respondents far from the mean of schooling (about 14.5 years) carry almost all of the weight; respondents at the mean carry essentially none. Know which observations are shouting.

4.3 Residual mechanics

```
e <- y - b0 - b1 * x
round(c(sum_e = sum(e), sum_xe = sum(x * e), cor_xe = cor(x, e)), 8)
#> sum_e sum_xe cor_xe
#>      0      0      0
```

These hold to machine precision in *every* sample, no matter how wrong the model is. They are consequences of the normal equations, not evidence of anything.

4.4 s^2 and the classical standard error

```
s2 <- sum(e^2) / (n - 2)
se_b1 <- sqrt(s2 / sum((x - mean(x))^2))
round(c(s2 = s2, se_b1 = se_b1), 5)
#>      s2      se_b1
#> 0.76066 0.00521
round(summary(fit_biv)$coefficients["educ", ], 5)
```

```
#> Estimate Std. Error t value Pr(>|t|)
#> 0.11873 0.00521 22.77854 0.00000
```

The standard error R prints is exactly $\sqrt{s^2 / \sum_i (X_i - \bar{X})^2}$, i.e. it *assumes homoskedasticity*. Next week we replace it.

5 Simulation: what the error assumption does

With real data we never know the truth. In a simulation we do, which makes it the honest way to see what an assumption buys.

5.1 When the average of Y really is a line

```
set.seed(1)
n_sim <- 2000
xs <- sample(8:20, n_sim, replace = TRUE) # schooling-like regressor
ys <- 8.2 + 0.12 * xs + rnorm(n_sim, 0, 0.7) # E[eps | x] = 0 by design
coef(lm(ys ~ xs)) |> round(4)
#> (Intercept) xs
#> 8.1642 0.1235
```

We wrote the truth ($\beta_0 = 8.2$, $\beta_1 = 0.12$) and OLS finds it, up to sampling noise. Rerun with a different seed: the estimate moves a little. How much it moves from sample to sample is precisely next week's topic.

5.2 When the CEF is not a line: uncorrelated is not mean-independent

Now make the population relationship curved, and fit a line anyway:

```
set.seed(2)
xq <- runif(n_sim, 8, 20)
yq <- 8 + 0.05 * xq + 0.015 * (xq - 14)^2 + rnorm(n_sim, 0, 0.4)
fq <- lm(yq ~ xq)

round(cor(xq, resid(fq)), 8) # exactly zero, mechanically
#> [1] 0

plot(xq, resid(fq), pch = 16, cex = .35, col = "navy",
     xlab = "x", ylab = "residual")
lines(lowess(xq, resid(fq)), col = "firebrick", lwd = 2)
abline(h = 0, lty = 2)
```

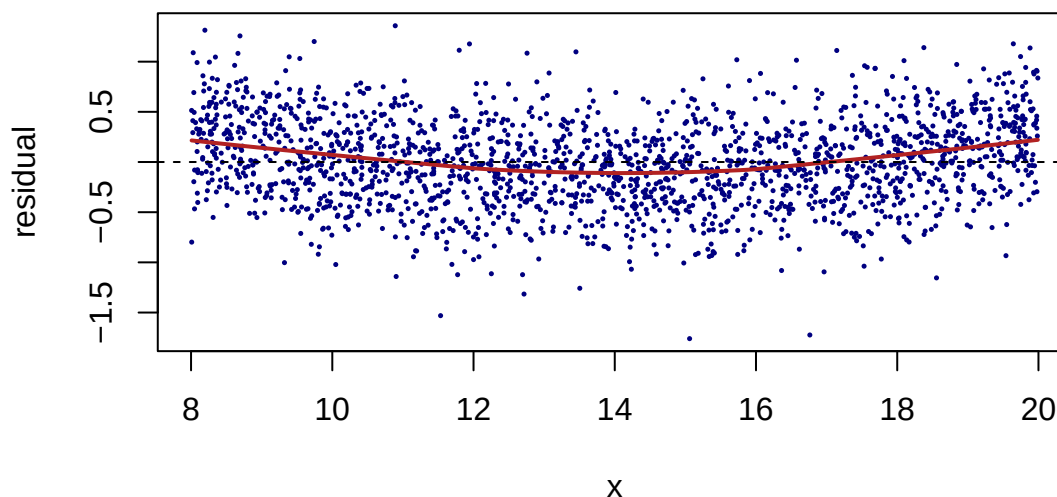


Figure 2: Residuals from fitting a line to a curved CEF. Their correlation with x is exactly zero, yet their average tracks the leftover curve: uncorrelated, but not mean-independent.

This is the ladder from lecture, live: the residuals are *uncorrelated* with x (bottom rung, guaranteed by the algebra) but their *average* clearly moves with x , so mean independence fails. The correlation could never have told you; the plot does. This is why you look at residual plots.

5.3 Heteroskedasticity does not bias the slope

```
set.seed(3)
slopes <- replicate(500, {
  xh <- runif(500, 8, 20)
  yh <- 8.2 + 0.12 * xh + rnorm(500, 0, 0.1 * xh) # spread grows with x
  coef(lm(yh ~ xh))[2]
})
round(c(mean_slope = mean(slopes), truth = 0.12), 4)
#> mean_slope      truth
#>    0.1209      0.1200
```

The error spread rises steeply with x , yet the slope estimates still centre on the truth: heteroskedasticity is a problem for *standard errors* (Week 2), not for the estimate itself. $E[\epsilon | X] = 0$ still holds here, and that is the assumption doing the work.

6 The multiple regression, and fit

With several regressors the same logic goes through; we let `lm()` carry the bookkeeping. (Thursday's session shows the notation that makes this compact.)

```
fit <- lm(llearn ~ educ + exper + I(exper^2) + female + black, data = gss)
y <- gss$llearn
round(coef(fit), 4)
```



```
#> (Intercept)      educ      exper  I(exper^2)      female      black
#>      7.3901      0.1405      0.0529      -0.0007     -0.3758     -0.1915
```

6.1 The sum-of-squares decomposition

```
yhat <- fitted(fit)
TSS <- sum((y - mean(y))^2)
ESS <- sum((yhat - mean(y))^2)
RSS <- sum(residuals(fit)^2)

c(TSS = TSS, ESS_plus_RSS = ESS + RSS)
#>      TSS ESS_plus_RSS
#>      3062      3062
c(R2_manual = ESS / TSS, R2_lm = summary(fit)$r.squared)
#> R2_manual      R2_lm
#>      0.2326      0.2326
```

And the model-comparison reading of R^2 : it is *PRE* against the intercept-only model.

```
fit_c <- lm(lnearn ~ 1, data = gss) # MODEL C
SSE_C <- sum(residuals(fit_c)^2)
SSE_A <- sum(residuals(fit)^2) # MODEL A
c(PRE = (SSE_C - SSE_A) / SSE_C, R2 = summary(fit)$r.squared)
#>      PRE      R2
#> 0.2326 0.2326
```

7 Frisch–Waugh–Lovell

We now verify FWL on the return to schooling: the treatment of interest is `educ`, and the controls are everything else in the regression.

```
## Step 1: residualize the treatment on the controls
d_tilde <- residuals(lm(educ ~ exper + I(exper^2) + female + black, data = gss))

## Step 2: residualize the outcome on the controls
y_tilde <- residuals(lm(lnearn ~ exper + I(exper^2) + female + black, data = gss))

## Step 3: bivariate regression of the residuals
b_fwl <- coef(lm(y_tilde ~ d_tilde))["d_tilde"]

c(full_regression = coef(fit)["educ"], fwl = b_fwl)
#> full_regression.educ      fwl.d_tilde
#>      0.1405      0.1405
```

Both forms of the regression-anatomy formula:

```
c(cov_ytilde_dtilde = cov(y_tilde, d_tilde) / var(d_tilde),
  cov_y_dtilde      = cov(gss$lnearn, d_tilde) / var(d_tilde))
#> cov_ytilde_dtilde      cov_y_dtilde
#>          0.1405          0.1405
```

In fact *all three* numerators agree, for the same reason:

```
c(both_residualized = cov(y_tilde, d_tilde) / var(d_tilde),
  only_regressor     = cov(gss$lnearn, d_tilde) / var(d_tilde),
  only_outcome       = cov(y_tilde, gss$educ) / var(d_tilde))
#> both_residualized      only_regressor      only_outcome
#>          0.1405          0.1405          0.1405
```

What you *cannot* change is the **denominator**:

```
c(correct = coef(lm(y_tilde ~ d_tilde))["d_tilde"],
  wrong   = coef(lm(y_tilde ~ gss$educ))[2])
#> correct.d_tilde      wrong.gss$educ
#>          0.1405          0.1293
```

i Note

Why does the second one fail? Regressing $\tilde{Y}$ on raw `educ` divides by $\text{Var}_n(\text{educ})$ instead of $\text{Var}_n(\widetilde{\text{educ}})$. Since residualizing can only shrink the variance, the denominator is too large and the estimate is attenuated. The numerators are all equal because the part removed from Y lies in the span of the controls and is orthogonal to $\tilde{D}$. Work this out on paper; it is Question 2 of Problem Set 1.

The residuals are also identical, as the theorem promises:

```
all.equal(unname(residuals(lm(y_tilde ~ d_tilde))), unname(residuals(fit)))
#> [1] TRUE
```

7.1 The added-variable plot

```
tibble::tibble(d_tilde, y_tilde) |>
  ggplot(aes(d_tilde, y_tilde)) +
  geom_point(alpha = .12, size = .5, colour = "navy") +
  geom_smooth(method = "lm", se = FALSE, colour = "firebrick", linewidth = .7) +
  labs(x = "Years of schooling | controls", y = "Log earnings | controls") +
  theme_minimal(base_size = 10)
```



Figure 3: Added-variable plot for years of schooling. The slope of the fitted line *is* the multivariate coefficient on `educ`.

Note how much of the spread in `educ` survives residualization ($\text{sd}(\tilde{d}) = 2.71$ versus $\text{sd}(\text{educ}) = 2.82$). When that ratio gets small, the coefficient is being estimated off a thin slice of the data, and its standard error will show it.

8 Omitted variable bias, numerically

Short regression omits verbal ability and parental education; long regression includes them.

```
long <- lm(lnearn ~ educ + exper + I(exper^2) + female + black +
          wordsum + pareduc, data = gss)
short <- lm(lnearn ~ educ + exper + I(exper^2) + female + black, data = gss)

## auxiliary regressions: each omitted variable on the included ones
aux_w <- lm(wordsum ~ educ + exper + I(exper^2) + female + black, data = gss)
aux_p <- lm(pareduc ~ educ + exper + I(exper^2) + female + black, data = gss)

beta_short <- coef(short)["educ"]
beta_long <- coef(long)["educ"]
bias_pred <- coef(long)["wordsum"] * coef(aux_w)["educ"] +
            coef(long)["pareduc"] * coef(aux_p)["educ"]

round(c(short = beta_short,
        long = beta_long,
```

```

    long_plus_bias = beta_long + bias_pred,
    bias = bias_pred), 5)
#>          short.educ          long.educ long_plus_bias.educ          bias.wordsum
#>          0.14053          0.11829          0.14053          0.02225

```

The identity $\tilde{\beta}_1 = \beta_1 + \sum_j \beta_{2j} \delta_{1j}$ holds exactly, to machine precision. Note the direction: ability and family background are positively associated with schooling and positively associated with earnings, so omitting them **overstates** the return to schooling.

```

modelsummary(list("(1) Bivariate" = lm(learn ~ educ, data = gss),
                  "(2) + demographics" = short,
                  "(3) + ability, background" = long),
              coef_map = c("educ" = "Years of schooling",
                           "wordsum" = "Vocabulary score",
                           "pareduc" = "Parental education"),
              gof_map = c("nobs", "r.squared"),
              stars = TRUE, output = "markdown")

```

	(1) Bivariate	(2) • demographics	(3) • ability, background
Years of schooling	0.119*** (0.005)	0.141*** (0.005)	0.118*** (0.006)
Vocabulary score			0.041*** (0.008)
Parental education			0.021*** (0.004)
Num.Obs.	3509	3509	3509
R2	0.129	0.233	0.245
• p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001			

! Important

Everything in this section is *algebra*. It tells you how the estimate moves when you add a control. It does **not** tell you whether the long regression recovers the causal return to schooling, for that you would need to argue that nothing else is omitted, which is a substantive claim, not a statistical one. That argument is what Weeks 5–11 are about.

9 Interpretation: functional form

9.1 Logs

With $\ln Y$ on the left, a coefficient β is approximately the proportional change in Y per unit of X . The exact percentage change is $100(e^\beta - 1)$.

```

b <- coef(long)["educ"]
c(approx_pct = 100 * b, exact_pct = 100 * (exp(b) - 1))
#> approx_pct.educ exact_pct.educ
#>          11.83          12.56

```

For small β the two agree; by $\beta = 0.5$ they do not (50% vs 64.9%).

9.2 Quadratics

`exper` enters quadratically, so the marginal effect depends on where you evaluate it:

```

b1 <- coef(long)["exper"]; b2 <- coef(long)["I(exper^2)"]
sapply(c(0, 10, 20, 30, 40), function(x) b1 + 2 * b2 * x) |>
  setNames(paste0("exper=", c(0, 10, 20, 30, 40))) |> round(4)
#> exper=0 exper=10 exper=20 exper=30 exper=40
#>  0.0541  0.0390  0.0238  0.0087 -0.0065

-b1 / (2 * b2)      # turning point, in years of experience
#> exper
#> 35.73

```

9.3 Interactions

```

fit_int <- lm(llearn ~ educ * female + exper + I(exper^2) + black +
              wordsum + pareduc, data = gss)
round(coef(fit_int)[c("educ", "female", "educ:female")], 4)
#>      educ      female educ:female
#>  0.1055   -0.7629    0.0270

```

The return to schooling is `educ` for men and `educ + educ:female` for women. Never read the `female` main effect as “the gender gap”: with an interaction it is the gap at `educ == 0`, which is outside the data.

9.4 Saturating the key covariate

```

fit_sat <- lm(llearn ~ factor(educ) + exper + I(exper^2) + female + black +
              wordsum + pareduc, data = gss)
c(linear_R2 = summary(long)$r.squared,
  saturated_R2 = summary(fit_sat)$r.squared)
#> linear_R2 saturated_R2
#>    0.2448    0.2633
anova(long, fit_sat)
#> Analysis of Variance Table
#>

```

```

#> Model 1: llearn ~ educ + exper + I(exper^2) + female + black + wordsum +
#>      pareduc
#> Model 2: llearn ~ factor(educ) + exper + I(exper^2) + female + black +
#>      wordsum + pareduc
#>      Res.Df  RSS Df Sum of Sq    F        Pr(>F)
#> 1      3501 2313
#> 2      3483 2256 18      56.7 4.86 0.000000000057 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

The F -test here is exactly the model comparison from *Social Statistics I*: is the reduction in SSE from letting every year of schooling have its own coefficient large enough to justify the extra parameters?

10 Exercises

Work these during lab; they are not collected, but Problem Set 1 builds directly on them.

1. **The dummy variable trap.** Run `lm(llearn ~ factor(region), data = gss)` and look at how many coefficients you get. Now try to include *all nine* region indicators and an intercept. What does `lm()` report, and why? Explain in words: the nine dummies add up to the intercept column, so one of them carries no new information.
2. **The $n - 2$, by simulation.** In the bivariate model, verify that $E[\sum_i \hat{e}_i^2] = \sigma^2(n - 2)$: fix `xs` from the first simulation, generate $y = 8.2 + 0.12x + \epsilon$ with $\epsilon \sim N(0, 4)$, and average $\sum_i \hat{e}_i^2$ over 2,000 replications. Compare to $4 \times (n - 2)$, and to the wrong guess $4 \times n$.
3. **FWL as fixed effects.** Regress `llearn` on `educ` and a full set of `region` dummies. Then (a) demean `llearn` and `educ` within `region` and regress the demeaned outcome on the demeaned regressor. Confirm the slopes match. This is the within estimator we will meet in Week 10.
4. **Sign the bias.** Suppose you omit `hours` from the long regression. Predict the sign of the bias in the `educ` coefficient *before* running anything, using the OVB formula and the two auxiliary regressions. Then check.
5. **Who carries the weight?** Using the weighted-slopes section, find the twenty observations with the largest weights w_i in the bivariate regression. What values of `educ` do they have? Refit without them, how much does the slope move? (Compare the move to the standard error before concluding anything.)
6. **The FWL standard error.** Compute $s^2 / \sum_i \tilde{d}_i^2$ using `d_tilde` from the FWL section (with s^2 from the full regression), take the square root, and confirm it matches the `educ` standard error printed by `summary(fit)`.

11 Session information

```
sessionInfo()
#> R version 4.5.2 (2025-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.5.2
#>
#> Matrix products: default
#> BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/vecLib.framework/Versions/A/vecLib.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] modelsummary_2.6.0 broom_1.0.12      ggplot2_4.0.2      dplyr_1.2.0
#>
#> loaded via a namespace (and not attached):
#> [1] Matrix_1.7-4      gtable_0.3.6      jsonlite_2.0.0
#> [4] compiler_4.5.2    tinytex_0.58      tidyselect_1.2.1
#> [7] tinytable_0.16.0  tidyr_1.3.2       splines_4.5.2
#> [10] scales_1.4.0      yaml_2.3.12       fastmap_1.2.0
#> [13] lattice_0.22-7    R6_2.6.1          labeling_0.4.3
#> [16] tables_0.9.33     generics_0.1.4    lmtest_0.9-40
#> [19] knitr_1.51        backports_1.5.0   checkmate_2.3.4
#> [22] tibble_3.3.1      insight_1.4.6     pillar_1.11.1
#> [25] RColorBrewer_1.1-3 rlang_1.1.7       performance_0.16.0
#> [28] xfun_0.56         parameters_0.28.3 S7_0.2.1
#> [31] datawizard_1.3.0  otel_0.2.0        cli_3.6.5
#> [34] withr_3.0.2       magrittr_2.0.4    mgcv_1.9-3
#> [37] digest_0.6.39     grid_4.5.2        nlme_3.1-168
#> [40] lifecycle_1.0.5   vctrs_0.7.1       evaluate_1.0.5
#> [43] glue_1.8.0        data.table_1.18.2.1 farver_2.1.2
#> [46] zoo_1.8-15        bayestestR_0.17.0 rmarkdown_2.30
#> [49] purrr_1.2.1       tools_4.5.2       pkgconfig_2.0.3
#> [52] htmltools_0.5.9
```